

OPEN SOURCE NODE.JS ARCHITECTURE

batch-rate-queue

The Zero-Redis Throttling & SQL Write-Buffering Layer. Built-in idempotency deduplication, partial batch failure isolation, PgBouncer-safe lease locks, multi-tenant fair scheduling, and PostgreSQL distributed coordination.

Package: batch-rate-queue Version: v1.4.0 Runtime: Node.js >= 16 License: MIT Tests: 161 / 161 Passing

1 Real Production Gaps in Existing Queue Systems

Traditional background job queues (BullMQ, pg-boss, Celery) were built to distribute compute across worker nodes. When modern applications interact with rate-limited APIs and relational databases, developers hit severe architectural walls:

1. Double-Firing External Side-Effects

When a worker crashes after calling an external API (Stripe, Twilio) but before marking the job done, redelivery calls the API a second time, duplicating charges or SMS.

2. All-or-Nothing Batch Rollbacks

When 1 row in a 50-row batch violates a constraint, Postgres rolls back the entire transaction by default, losing the 49 good writes along with the bad one.

3. PgBouncer Transaction Mode Breakage

Managed databases (Supabase, Neon, RDS Proxy) run PgBouncer in transaction-pooling mode, which silently breaks session-scoped pg_advisory_lock.

4. Multi-Tenant Starvation & Paid Paywalls

One heavy tenant enqueueing 50k jobs blocks everyone else. BullMQ removed group rate limiting in v3 and made it paid-only. No free, robust multi-tenant scheduler existed.

2 The "Zero-Redis" Architectural Paradigm

The Dedicated Throttling & Write-Buffering Layer

Instead of replacing your existing orchestrator, batch-rate-queue acts as the specialized throttling and write-buffering layer that sits in front of whatever executes your jobs. It runs purely in-process with optional PostgreSQL distributed coordination across pods—eliminating Redis entirely.

0

REDIS CLUSTERS NEEDED

98.5%

DB ROUNDTRIP REDUCTION

100%

TENANT STARVATION FREE (DRR)

0B

DATA LOST ON SIGTERM

3 Complete Architecture Blueprint

Incoming tasks flow through a multi-stage pipeline guaranteeing fair scheduling, token budgeting, idempotency deduplication, adaptive backoff, and transactional batch writing:



4 Six Pillar Capabilities

1. Idempotency & Deduplication
Prevents duplicate external API calls (e.g. double-charging Stripe) on worker redeliveries after unhandled container crashes.

2. Partial Batch Failure Isolation
When 1 row in a 50-row batch fails a constraint, automatically isolates the bad row and commits the other 49 valid rows.

3. PgBouncer-Safe Lease Locks
Distributed lock table using UTC timestamp intervals, 100% safe under Supabase/Neon transaction pooling mode.

4. Multi-Tenant Fair-Share (DRR)
Deficit Round Robin scheduling isolates tenant queues. A tenant with 50,000 items will never starve a tenant with 5 items.

5. Multi-Provider Header Parsing
Parses IETF RFC 9745, OpenAI, Anthropic, GitHub, Vercel, and HTTP-Date Retry-After headers automatically.

6. Payload Claim-Check Pattern
Automatically offloads payloads >64KB to prevent DB table bloat and preserve high-throughput SKIP LOCKED scans.

5 Comprehensive Feature Comparison

Feature Capability	BullMQ	pg-boss	batch-rate-queue
Redis Dependency	Mandatory Redis Cluster	Zero (Postgres only)	Zero (Standalone or Postgres)
Partial Batch Failure Isolation	No (All or nothing)	No (All or nothing)	Native Row-Level Salvage Isolation
Side-Effect Idempotency	Manual only	Manual only	Built-in IdempotencyManager
PgBouncer Transaction Mode	N/A (Redis)	Advisory locks break in tx mode	100% Safe (PgLeaseLock Table Rows)
Per-Key Multi-Tenant Isolation	Paid-only Pro tier	No native per-key buckets	Built-in KeyedThrottler (Free/MIT)
Fair-Share Tenant Scheduling	FIFO single queue	FIFO single queue	Deficit Round Robin (DRR) Fair Share
Distributed Circuit Breaker	Requires external tool	None	Postgres Shared Circuit Breaker
Claim-Check Large Payloads	Manual	Manual	Automatic Offload & Hydration

6 Code Recipe: Side-Effect Idempotency Deduplication

Prevent duplicate external charges or notifications when workers crash before marking a job complete:

```
idempotent-stripe-worker.ts TypeScript

import { createBatchRateQueue } from 'batch-rate-queue';

const queue = createBatchRateQueue({
  name: 'billing-charges',
  rateLimit: { requests: 20, perMs: 1000 },
  batchFlush: { size: 50, intervalMs: 2000 },

  // Deduplicate by charge idempotency key (24-hour cache)
  idempotencyKey: (item) => `charge_${item.orderId}`,
  idempotencyTtlMs: 24 * 60 * 60 * 1000,

  worker: async (item) => {
    const charge = await stripe.charges.create({
      amount: item.amount,
      currency: 'usd',
    });
    return { id: item.id, updates: { chargeId: charge.id, status: 'paid' } };
  },
  onBatchFlush: async (batch) => db.saveCharges(batch),
});
```

7 Code Recipe: LLM Token-Budget Limiting (TPM)

Enforce token-based budgets (e.g. 40,000 tokens/min) for OpenAI or Anthropic API calls with pre-acquisition and actual-cost reconciliation:

```
llm-embeddings-worker.ts TypeScript

import { createBatchRateQueue, llmApiClassifier } from 'batch-rate-queue';
import OpenAI from 'openai';

const openai = new OpenAI();

const queue = createBatchRateQueue({
  name: 'openai-embeddings',
  // Budget: 40,000 tokens per minute
  rateLimit: { requests: 40000, perMs: 60000, costBased: true },
  batchFlush: { size: 50, intervalMs: 3000 },

  // Estimate token cost before acquire
  costExtractor: (item) => Math.ceil(item.text.length / 4),
  errorClassifier: llmApiClassifier,

  worker: async (item) => {
    const res = await openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: item.text,
    });
    return {
      id: item.id,
      updates: { embedding: res.data[0].embedding },
      meta: { actualCost: res.usage.total_tokens }, // Exact actual cost
    };
  },
  onBatchFlush: async (batch) => db.saveVectors(batch),
});
```

8 Code Recipe: PgBouncer-Safe Distributed Lease Locks

Execute distributed scheduled jobs safely under PgBouncer transaction-pooling mode (Supabase, Neon, RDS Proxy):

pgbouncer-lease-lock.ts

TypeScript

```
import { PgLeaseLock } from 'batch-rate-queue';
import { Pool } from 'pg';

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

const lock = new PgLeaseLock({
  pool,
  lockKey: 'cron:daily-reconciliation',
  ttlMs: 30000, // 30-second lease with automatic heartbeat renewal
});

// Automatically acquires, heartbeats, executes, and releases safely
await lock.runWithLock(async () => {
  console.log('Running daily reconciliation with guaranteed single-replica execution!');
  await reconcileAccounts();
});
```

9 Code Recipe: Multi-Tenant Fair Scheduling (DRR)

Isolate rate limits per tenant and eliminate tenant starvation via Deficit Round Robin:

multi-tenant-fair-share.ts

TypeScript

```
const queue = createBatchRateQueue({
  name: 'multi-tenant-sync',
  rateLimit: { requests: 10, perMs: 1000 },
  batchFlush: { size: 100, intervalMs: 2000 },
  rateLimitKey: (item) => item.tenantId,
  perKeyRateLimit: {
    'enterprise-corp': { requests: 100, perMs: 1000 },
    'free-tier-user': { requests: 2, perMs: 1000 },
  },
  // DRR: 10k items from tenant A will never starve tenant B
  fairShare: true,
  worker: async (item) => syncTenantData(item),
  onBatchFlush: async (batch) => db.bulkUpdate(batch),
});
```

10

Native Database Adapters

Prisma Adapter

```
import { createPrismaFlushHandler }
  from 'batch-rate-queue';

const onFlush =
  createPrismaFlushHandler(
    prisma, 'location'
  );
```

PostgreSQL (pg)

```
import { createPgFlushHandler }
  from 'batch-rate-queue';

const onFlush =
  createPgFlushHandler(
    pool, 'locations'
  );
```

SQLite

```
import { createSqliteFlushHandler }
  from 'batch-rate-queue';

const onFlush =
  createSqliteFlushHandler(
    db, 'locations'
  );
```

11

Full Options Reference Table

Property	Type	Default	Description
name	string	required	Queue identifier for logs, metrics, and default bucket keys.
rateLimit	RateLimitConfig	required	Token bucket rate limit (requests, perMs, costBased).
batchFlush	BatchFlushConfig	required	Transactional write buffer trigger (size, intervalMs).
worker	Function	required	Worker execution per item returning { id, updates, meta }.
onBatchFlush	Function	required	Atomic transaction write callback (or pre-built DB adapter).
idempotencyKey	(item) => string	undefined	Extracts deduplication key to prevent duplicate external side effects.
isolateBatchFailures	boolean	true	Salvages 49 valid rows when 1 row fails a constraint in a 50-item batch.
claimCheckThresholdBytes	number	65536	Offloads payloads >64KB to preserve fast DB index scans.
rateLimitKey	(item) => string	undefined	Isolates token buckets per tenant, API key, or destination.
fairShare	boolean	false	Enables Deficit Round Robin scheduling across tenant queues.
costExtractor	(item) => number	() => 1	Pre-acquire token budget estimator (for LLM tokens/TPM).
distributed	DistributedConfig	undefined	PostgreSQL-backed shared token bucket (zero-Redis multi-pod).
circuitBreaker	CircuitBreakerConfig	undefined	PostgreSQL-backed cross-process failure circuit breaker.

12

Official Ecosystem Resources

- **npm:** <https://www.npmjs.com/package/batch-rate-queue>
- **GitHub:** <https://github.com/piyushkumar-prog/batch-rate-queue>
- **License:** MIT (Open Source, Free for Commercial Use)
- **Author:** Piyush Kumar (@piyushkumar-prog)